

# Intro to Deep Learning

## Lecture 02: Backpropagation, Optimizers, and Training Loops

Stanislav Don

DS212 - Intro to Deep Learning

# Today

By the end of the lecture, you should be able to answer:

1. How does a neural network learn from a loss?
2. What is a computational graph?
3. What does backpropagation compute?
4. Why do we train with mini-batches?
5. How do common optimizers differ?
6. What does a clean training loop look like?

Lecture rhythm: scalar example -> graph -> PyTorch -> training loop

# Recap From Lecture 01

A neural network is a trainable function:

$$\hat{y} = f_{\theta}(x)$$

Lecture 01 focused on:

```
input -> model -> prediction -> loss
```

Today we close the loop:

```
loss -> gradients -> optimizer step -> updated model
```

## Part 1

# From Loss To Learning

Training is numerical optimization

# The Training Problem

We have:

- data:  $(x_i, y_i)$
- model:  $f_\theta$
- loss function:  $L$

We want parameters  $\theta$  that make predictions good:

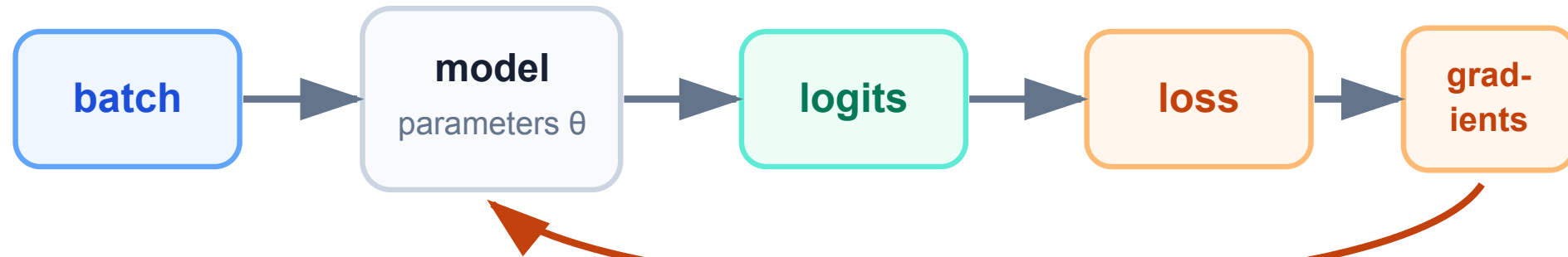
$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_\theta(x_i), y_i)$$

Training is the process of changing  $\theta$  to reduce this objective.

# The Big Loop

Almost every supervised training loop has the same skeleton.

**One iteration: forward, backward, update**



backprop computes gradients for model parameters  
optimizer.step() changes  $\theta$  using those gradients

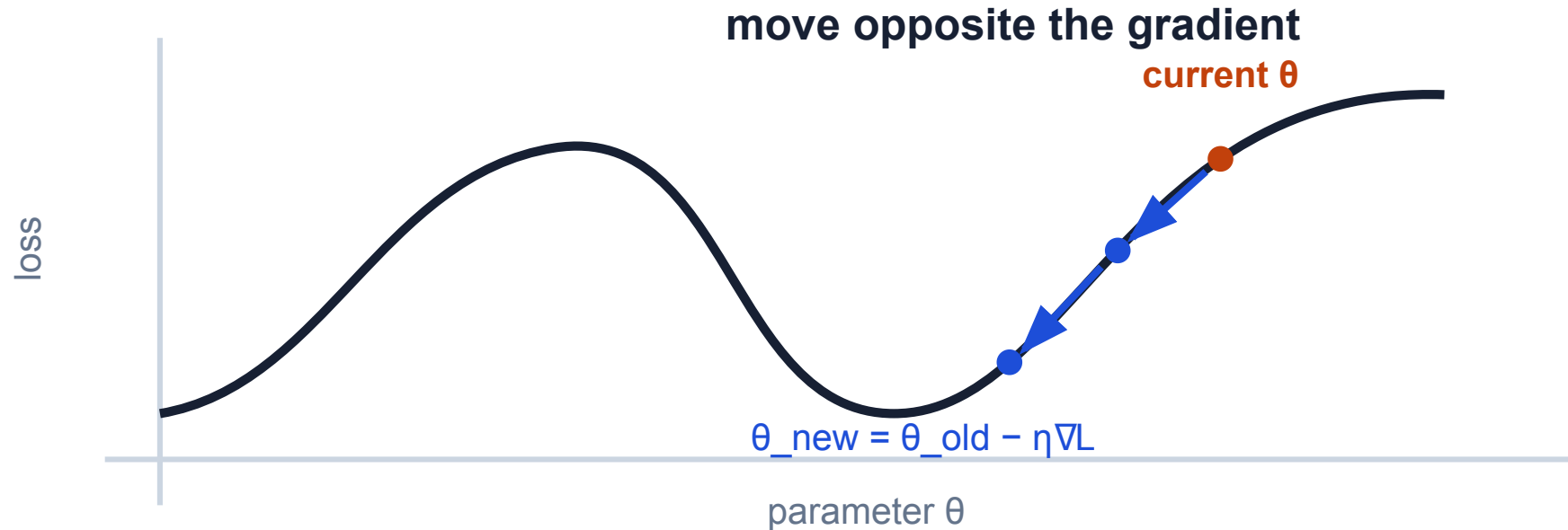
Everything else is detail, discipline, and debugging.

# Gradient Descent Intuition

The gradient points in the direction where the loss increases fastest.

So we move in the opposite direction:

$$\theta_{new} = \theta_{old} - \eta \nabla_{\theta} L$$



# Learning Rate

The learning rate  $\eta$  controls step size.

| If $\eta$ is... | Training often looks like...       |
|-----------------|------------------------------------|
| too small       | slow progress, loss barely changes |
| reasonable      | loss decreases, then plateaus      |
| too large       | jumps, divergence, NaN             |

Learning rate is often the first hyperparameter to check.



**Part 2**

# **Backpropagation**

**Efficient chain rule on a graph**

# A Tiny Example

Let:

$$f(a) = (3a + 2)^2$$

Define:

$$u = 3a + 2$$

$$f = u^2$$

We can compute the derivative using the chain rule.

# Chain Rule

$$\frac{df}{da} = \frac{df}{du} \cdot \frac{du}{da}$$

For our example:

$$f = u^2$$

$$\frac{df}{du} = 2u$$

$$u = 3a + 2$$

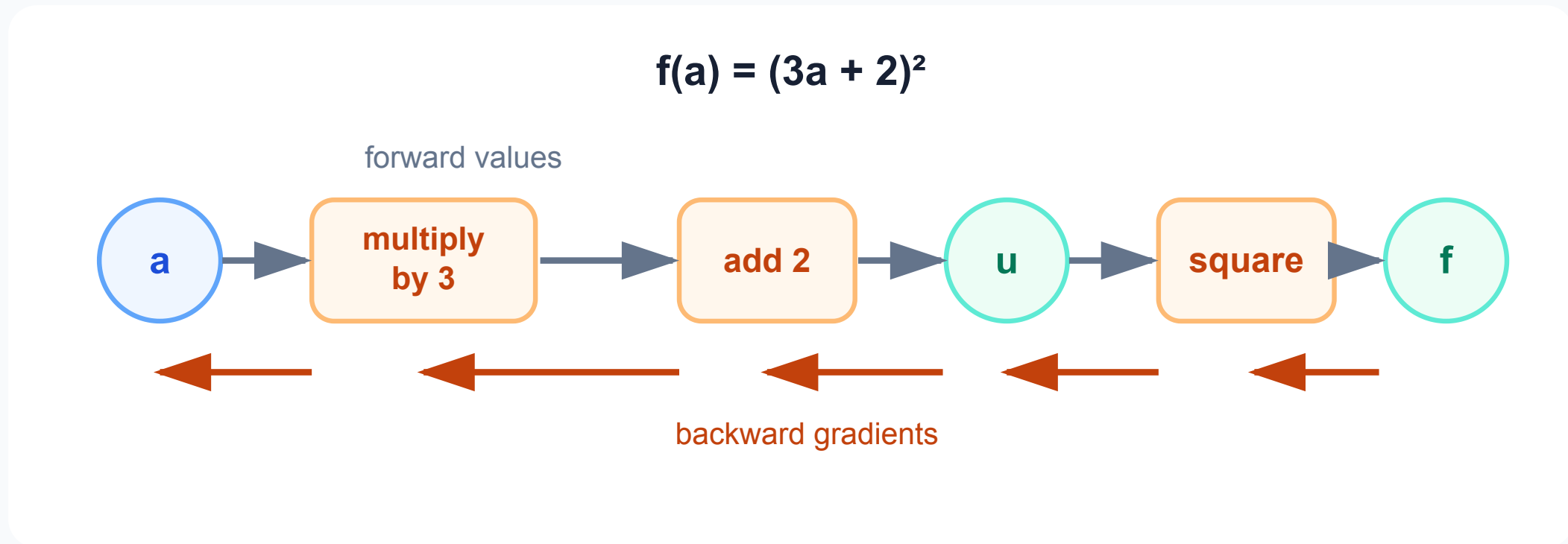
$$\frac{du}{da} = 3$$

Therefore:

$$\frac{df}{da} = 2(3a + 2) \cdot 3$$

# Computational Graph

The same example as a graph:



Forward pass computes values.

Backward pass computes gradients.

# Local Derivatives

Backprop works because each operation knows its local derivative.

| Operation            | Local derivative                                                       |
|----------------------|------------------------------------------------------------------------|
| $z = x + y$          | $\frac{\partial z}{\partial x} = 1, \frac{\partial z}{\partial y} = 1$ |
| $z = xy$             | $\frac{\partial z}{\partial x} = y, \frac{\partial z}{\partial y} = x$ |
| $z = x^2$            | $\frac{\partial z}{\partial x} = 2x$                                   |
| $z = \text{ReLU}(x)$ | 1 if $x > 0$ , else 0                                                  |

Frameworks combine these local derivatives automatically.

# One Useful Mental Model

Each operation receives:

gradient of loss with respect to its output

and computes:

gradient of loss with respect to its inputs

This is why layers can be stacked: each layer only needs to know its local part.

## Part 3

# PyTorch Autograd

The graph is built during the forward pass

# What PyTorch Stores

If a tensor has `requires_grad=True`, PyTorch tracks operations that use it.

```
x = torch.tensor(2.0, requires_grad=True)
y = (3 * x + 2) ** 2
```

For a computed tensor:

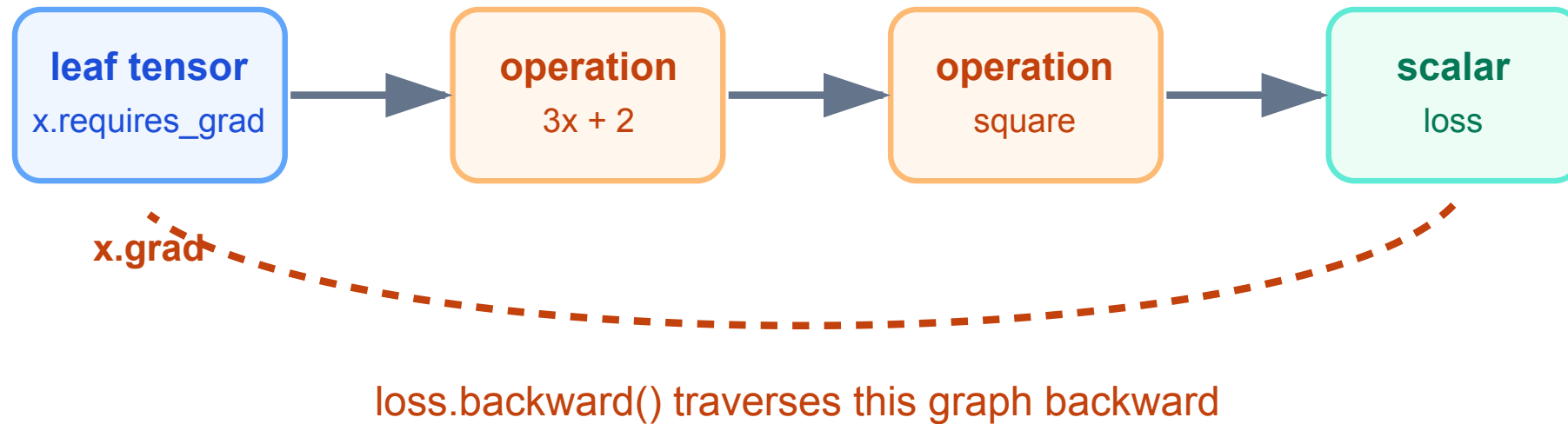
```
y.grad_fn
```

points to the graph node that created it.



# Autograd Graph

Autograd records operations, then fills `.grad` on leaves



PyTorch does not store gradients for every intermediate tensor by default.

The usual goal is to fill `.grad` for trainable parameters.

# Calling `backward()`

When we call `backward()` on a scalar loss:

```
loss.backward()
```

PyTorch:

1. traverses the graph backward
2. applies chain rule
3. fills `.grad` for trainable leaf tensors

The graph was created by the forward pass that produced this loss.

# Leaf Tensors

Leaf tensors are tensors created directly by the user.

Usually trainable parameters are leaf tensors.

```
w = torch.randn(3, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
```

After:

```
loss.backward()
```

we inspect:

```
w.grad
b.grad
```

# Practical PyTorch Rule

Before computing new gradients, clear old ones:

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Why this matters:

```
PyTorch adds new gradients into .grad
```

If you forget `zero_grad()`, updates use old gradients plus new gradients.

## Part 4

# Backprop Through An MLP

**Gradients flow through the same graph as values**

# A 2-Layer MLP

Consider:

$$z_1 = W_1x + b_1$$

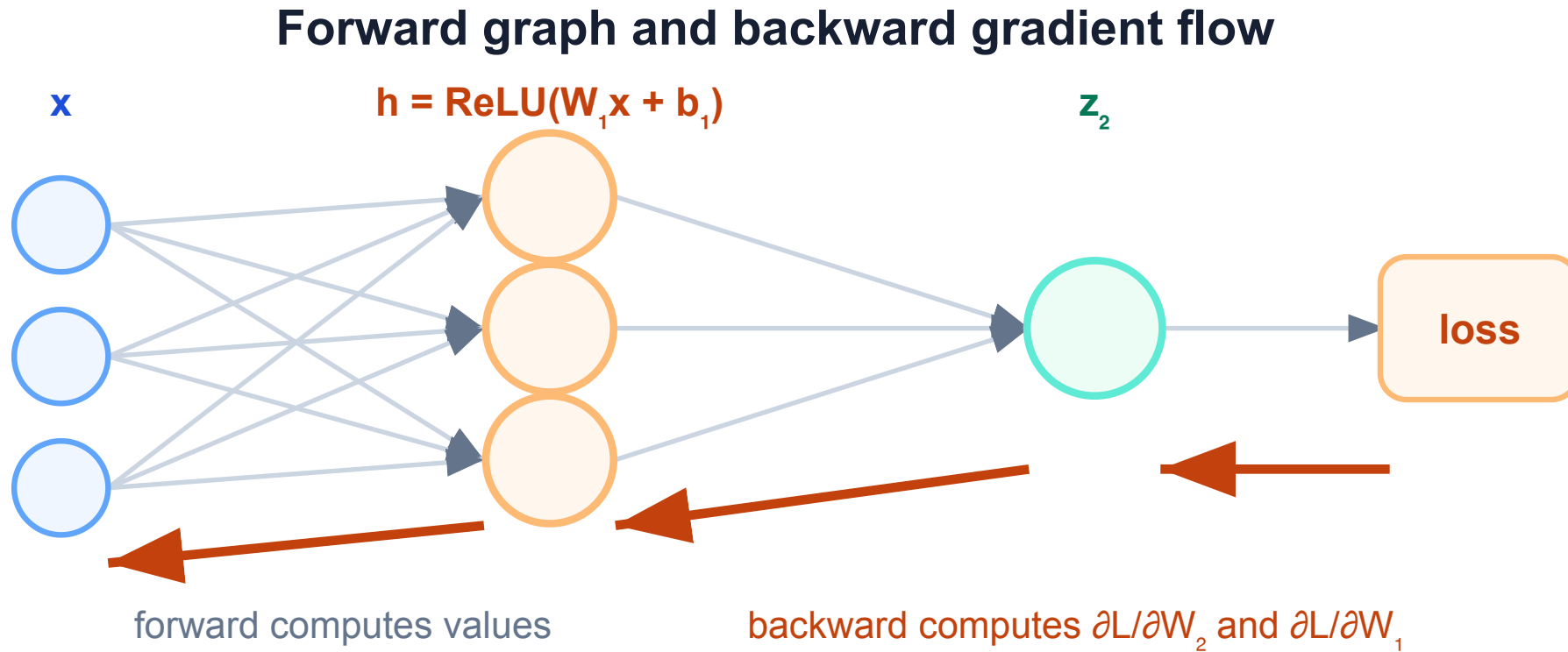
$$h = \text{ReLU}(z_1)$$

$$z_2 = W_2h + b_2$$

$$L = \text{loss}(z_2, y)$$

We want gradients for  $W_1$ ,  $b_1$ ,  $W_2$ , and  $b_2$ .

# Forward And Backward Flow



The loss is a scalar, but its gradient flows back to many parameters.

# Forward And Backward Values

Forward pass computes:

```
x  
z1 = w1 x + b1  
h  = ReLU(z1)  
z2 = w2 h + b2  
L  = loss(z2, y)
```

Backprop computes:

```
dL/dw2, dL/db2  
dL/dw1, dL/db1
```

It does this by moving backward through the same graph.

The loss depends directly on  $z_2$ :



# ReLU In Backprop

For:

$$h = \text{ReLU}(z_1)$$

The derivative is:

$$\frac{dh}{dz_1} = \begin{cases} 1, & z_1 > 0 \\ 0, & z_1 \leq 0 \end{cases}$$

If a hidden unit was inactive in the forward pass, it blocks gradient in the backward pass.

# Matrix Calculus Borderline

For this course, you should understand:

- which tensors influence the loss
- which gradients are computed
- how shapes should match
- why chain rule is enough

You do not need to manually derive every matrix gradient.

But you should be able to trace:

```
loss -> output -> hidden layer -> earlier parameters
```

## Part 5

# Mini-Batch Training

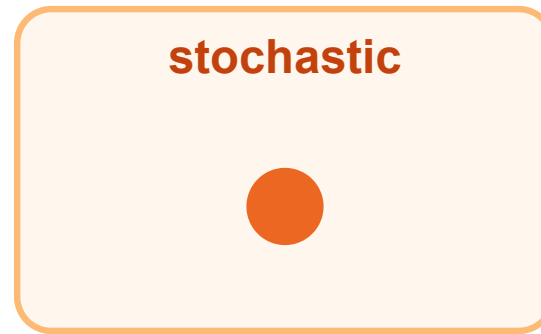
The standard unit of deep learning optimization

# Batch Choices

## Three ways to estimate a gradient



stable, expensive



cheap, very noisy



standard DL choice

Mini-batches are the standard compromise:

- cheaper than full-dataset updates
- less noisy than one-sample updates

# Mini-Batch Loss

For one batch of size  $B$ :

$$L_{batch} = \frac{1}{B} \sum_{i=1}^B L_i$$

Then one optimizer step:

```
compute batch loss  
compute gradients  
update parameters
```

This is the standard deep learning setup.

# Why Shuffle Training Data?

Without shuffling, batches may be biased.

Example:

```
first 1000 examples: class 0  
next 1000 examples: class 1
```

Training batches would be class-pure.

With shuffling, each batch is more representative.

## Part 6

# Optimizers

Same gradients, different update rules

# Optimizer Progression

Optimizer ideas build on the same gradient



raw step



memory



adaptive scale



memory + scale



decoupled decay

Optimizer choice matters, but learning rate usually matters first.

All of them use gradients.

They differ in how they turn gradients into parameter updates.



# Optimizer 1: SGD

SGD update:

$$\theta_{t+1} = \theta_t - \eta g_t$$

Where:

$$g_t = \nabla_{\theta} L_{batch}$$

SGD is simple and strong, especially for some vision models.

It can be sensitive to learning rate and may need momentum.

# Optimizer 2: Momentum

Momentum keeps a running velocity:

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Intuition:

- smooth noisy gradients
- build speed in consistent directions
- reduce zig-zagging

## Optimizer 3: RMSProp

RMSProp keeps a running average of squared gradients:

$$s_t = \beta s_{t-1} + (1 - \beta)g_t^2$$

Then it scales the update:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

Parameters with consistently large gradients get smaller relative steps.

# Optimizer 4: Adam And AdamW

Adam combines:

- momentum-like running average of gradients
- RMSProp-like running average of squared gradients

Practical intuition:

Adam adapts step sizes per parameter  
and smooths noisy gradients

It is a strong default optimizer for many tasks.

AdamW is Adam with a better treatment of weight decay.

For modern deep learning, is often a good default.

# Practical Optimizer Advice

Good starting points:

| Situation                    | Common starting point   |
|------------------------------|-------------------------|
| small MLP / quick experiment | Adam or AdamW           |
| CNN from scratch             | AdamW or SGD + momentum |
| transformer fine-tuning      | AdamW                   |

Most important:

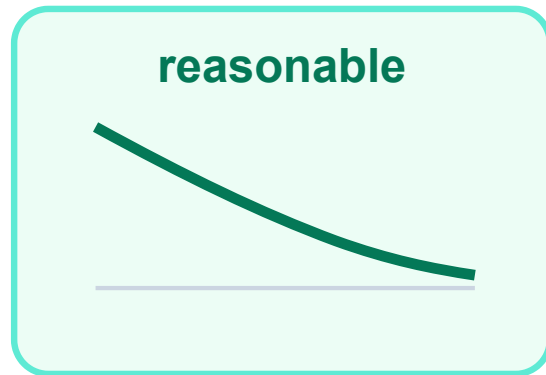
tune learning rate before trying exotic optimizers

# Learning Rate Symptoms

Learning-rate symptoms show up in the loss curve



barely moves



decreases, then plateaus



unstable or NaN

Schedulers come in Lecture 04.

For now: learn to read the loss curve before changing the model.

## **Part 7**

# **Clean Training And Evaluation**

**Most training bugs are workflow bugs**

# Clean Training Step

One training iteration:

```
model.train()

optimizer.zero_grad()
logits = model(batch_x)
loss = loss_fn(logits, batch_y)
loss.backward()
optimizer.step()
```

The order matters.

Forgetting one line is enough to break training.



# Why `model.train()` ?

Some layers behave differently during training and evaluation.

Examples:

- Dropout
- BatchNorm

During training:

```
model.train()
```

During validation/testing:

```
model.eval()
```

Even if today's model does not need it, build the habit now.

# Clean Evaluation Step

Evaluation:

```
model.eval()

with torch.no_grad():
    logits = model(batch_x)
    loss = loss_fn(logits, batch_y)
```

Why `torch.no_grad()` ?

- saves memory
- speeds up evaluation
- avoids building unnecessary graphs

We do not update parameters during evaluation.

# Binary Accuracy From Logits

For binary classification with one output logit:

```
logits.shape == (B,)  
targets.shape == (B,)
```

Prediction:

```
preds = (logits >= 0).float()
```

Why threshold at 0?

$$\text{sigmoid}(\text{logit}) \geq 0.5 \iff \text{logit} \geq 0$$

No need to apply sigmoid just to classify.

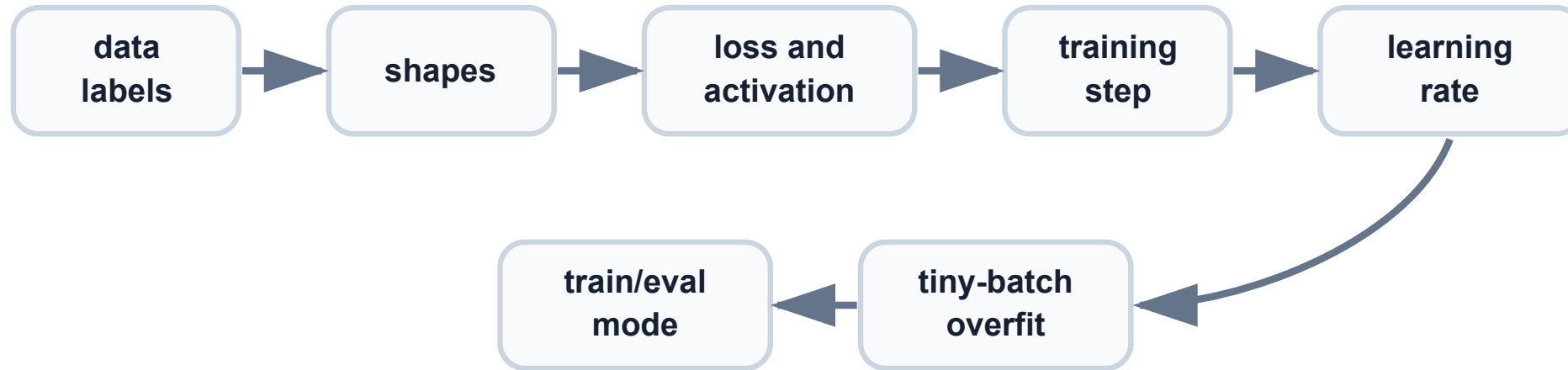
## Part 8

# Debugging Training

Start with the boring checks

# It Does Not Learn

**Debug the boring things first**



Most training bugs are boring. That is good news.

Boring bugs are findable.

# Data And Shape Checks

Ask about data:

- Are labels correct?
- Are labels aligned with inputs?
- Are train and validation distributions similar?
- Did preprocessing accidentally destroy signal?

Common binary shape bug:

```
logits: (B, 1)
targets: (B,)
```

# Loss And Activation Match

For binary classification:

```
model outputs raw logits  
loss_fn = nn.BCEWithLogitsLoss()
```

Risky / less stable:

```
model ends with Sigmoid()  
loss_fn = nn.BCELoss()
```

For multiclass classification:

```
model outputs raw logits  
loss_fn = nn.CrossEntropyLoss()
```

No softmax before `CrossEntropyLoss` .

# Optimizer Step Bugs

These all break or distort training:

```
# forgot this  
optimizer.zero_grad()
```

```
# forgot this  
loss.backward()
```

```
# forgot this  
optimizer.step()
```

Symptoms:

- no learning
- exploding gradients
- unstable updates



# Sanity Checks

Useful small tests:

1. Can the model overfit a tiny batch?
2. Does loss decrease on one batch?
3. Are gradients non-zero?
4. Do parameters change after `optimizer.step()` ?
5. Is a simple baseline competitive?

If a model cannot overfit 32 examples, something is probably wrong.

# What Will Happen In Seminar?

You will:

1. inspect a PyTorch computational graph
2. compare manual derivatives with autograd
3. create a custom `Dataset`
4. train with `DataLoader`
5. write `train_one_epoch`
6. write `evaluate`
7. use a debugging checklist when training fails

This is the engineering skeleton for the whole course.

# Mini Check

What is wrong with this training step?

```
logits = model(x)
loss = loss_fn(logits, y)
optimizer.step()
optimizer.zero_grad()
```

# Answer

There is no:

```
loss.backward()
```

So gradients were never computed for the current loss.

Correct pattern:

```
optimizer.zero_grad()  
logits = model(x)  
loss = loss_fn(logits, y)  
loss.backward()  
optimizer.step()
```

# Mini Check

Why do we use:

```
with torch.no_grad():  
    ...
```

during validation?

# Answer

During validation:

- we do not update parameters
- we do not need gradients
- we do not need to build a computational graph

So `torch.no_grad()` saves memory and computation.

It also makes our intent clear.

# Key Takeaways

1. Backpropagation is efficient chain rule on a computational graph.
2. `loss.backward()` computes gradients.
3. Optimizers use gradients to update parameters.
4. Mini-batches are the standard unit of DL training.
5. Clean training/evaluation loops prevent many bugs.
6. When training fails, debug boring things first.

Next lecture:

classification losses, logits, softmax, and evaluation

## During Seminar 02, keep asking:

```
What graph did PyTorch build?  
Which tensors receive gradients?  
Did we zero gradients before backward?  
Are logits and targets the same shape?  
Are we in train mode or eval mode?  
Does the loss decrease?
```

If those questions become automatic, the rest of the course becomes much easier.